low complexity, performs well, and you should be fine calling it very frequently.

As a comparison, Sidekiq uses LRANGE when iterating over queues.[20] The complexity of this command is O(S + N). Both S and N are documented, though in Big O Notation, N is usually a placeholder for "the size of the data being operated on." In this case, the cost of calling LRANGE will increase linearly with the number of queues you have. It still may be very fast, but it's an order of magnitude slower than SCARD, all things being equal.

Of course, these are just estimates, but they still give you some insights into what will happen if you call particular Sidekiq API methods frequently. After tracing the API calls to Redis commands a few times, you'll find you can quickly look up any other call and clearly understand what you are doing before you do it. This will prevent you from inadvertently tanking your app's performance by trying to monitor it too aggressively.

This all might seem really low level, but you do need to know how Redis works to understand the performance of your Sidekiq installation.

Understand How Redis Works

Redis is an in-memory data store. If Redis were to restart, or the machine it's running on restarts or goes away (as may happen in a cloud computing environment), all the data is lost. Since that data represents your unexecuted jobs, that means they would be lost. If that happens, to quote the inimitable Egon Spengler, "It would be bad."

Fortunately, there are many ways to configure Redis in production to vastly reduce the chance of this happening. While you may not need a full-time Redis expert to *operate* Redis, you should avail yourself of one to set it up, especially if you are doing it yourself or using something like Amazon Web Service's (AWS) Elasticache, since there aren't any presets tailored to Sidekiq.